

## SESSION 6

### Django Forms Basics

**Topics Covered:** Django Form class, GET vs POST, CSRF token, Handling form submission

**Activities:** Build 'Add Doctor' form, Display validation errors

**Task 1: Create a Django Form class named AddRestaurantForm with fields for restaurant name, cuisine type, and contact email, then render it in a template using the GET method.**

**Explanation & Approach:** *Django Forms handle rendering HTML controls, parsing input data, and applying automated server-side validations.*

```
# foodiespot/forms.py
from django import forms

class AddRestaurantForm(forms.Form):
    restaurant_name = forms.CharField(label="Restaurant Name", max_length=100)
    cuisine_type = forms.CharField(label="Cuisine Type", max_length=50)
    contact_email = forms.EmailField(label="Contact Email")
```

**Task 2: Modify AddRestaurantForm to handle form submission using POST method and display submitted data on a new page after user submits form.**

**Explanation & Approach:** *HTTP POST method securely transmits user inputs in the request body rather than visible URL parameters.*

```
# foodiespot/views.py
from django.shortcuts import render
from .forms import AddRestaurantForm

def add_restaurant_view(request):
    if request.method == 'POST':
        form = AddRestaurantForm(request.POST)
        if form.is_valid():
            # Process cleaned data
            data = form.cleaned_data
            return render(request, 'foodiespot/success.html', {'data': data})
        else:
            form = AddRestaurantForm()

    return render(request, 'foodiespot/add_restaurant.html', {'form': form})
```

**Task 3: Add validation to AddRestaurantForm so restaurant name must be at least 3 characters and contact email valid. Display validation errors directly above form fields.**

**Explanation & Approach:** *Custom field cleaning methods (clean\_fieldname) enforce domain business validation rules before form processing.*

```
# foodiespot/forms.py (With Validation)
from django import forms
from django.core.exceptions import ValidationError

class AddRestaurantForm(forms.Form):
    restaurant_name = forms.CharField(label="Restaurant Name", max_length=100)
    cuisine_type = forms.CharField(label="Cuisine Type", max_length=50)
    contact_email = forms.EmailField(label="Contact Email")

    def clean_restaurant_name(self):
        name = self.cleaned_data.get('restaurant_name')
        if len(name) < 3:
            raise ValidationError("Restaurant name must be at least 3 characters long.")
        return name
```

#### Task 4: Implement CSRF protection in your form template and verify that submitting form without CSRF token results in forbidden error.

**Explanation & Approach:** *Django requires {% csrf\_token %} inside POST forms to prevent Cross-Site Request Forgery security exploits.*

```
<!-- templates/foodiespot/add_restaurant.html -->
{% extends 'base.html' %}

{% block content %}
<div class="row justify-content-center">
    <div class="col-md-6">
        <h3 class="mb-3 font-weight-bold">Add New Restaurant</h3>

        {% if form.errors %}
        <div class="alert alert-danger">
            <strong>Please correct the errors below:</strong>
            {{ form.errors }}
        </div>
        {% endif %}

        <form method="POST" action="">
            {% csrf_token %}

            <div class="mb-3">
                {{ form.restaurant_name.label_tag }}
                {{ form.restaurant_name }}
            </div>
            <div class="mb-3">
                {{ form.cuisine_type.label_tag }}
                {{ form.cuisine_type }}
            </div>
            <div class="mb-3">
                {{ form.contact_email.label_tag }}
                {{ form.contact_email }}
            </div>
        </form>
    </div>
</div>
```

```
        <button type="submit" class="btn btn-primary">Submit Restaurant</button>
      </form>
    </div>
  </div>
{% endblock %}
```